Fertile Abstractions:
Measuring How a Learned Idea Changes Later Discovery

PIATRA . INSTITUTE

October 2026

Abstract

A learned abstraction can change what a program-searching learner discovers afterwards. We define its discovery value as the difference in a frozen learner's success on withheld tasks between learning with the abstraction available and learning without it, with the abstraction supplied at the test in both cases. Adding an abstraction defined from existing primitives never changes what can be expressed, only what a bounded search reaches. We prove that discovery value is not determined by compression of past solutions, direct usefulness or the immediate effect on search, and that two abstractions can exchange their order by discovery value when checking candidates becomes more expensive, so no score that ignores the search budget orders abstractions correctly at every cost. In 300 random worlds in each of two domains, list programs and navigation between rooms, discovery value varied more across abstractions than direct value did (standard deviations 0.096 and 0.042 in the share of test tasks solved; 0.053 and 0.037), took both signs, was reliable across independent test families and had the opposite sign to direct value for more than half of the abstractions. Within worlds, compression of past solutions correlated with it at 0.11 and 0.08 and a look-ahead on the learner's own curriculum at 0.22 and 0.08, against reliability ceilings of 0.88 and 0.80. Rankings at verification costs eight times apart agreed at only 0.16 and 0.19 after correction for reliability. Fertility here belongs to an abstraction together with a budget and a future, and is mostly invisible when the abstraction is adopted.

1. Two abstractions with the same record

A learner that solves problems by searching over programs can name a recurring piece of its solutions and use the name as a single step afterwards. Suppose two such pieces appear equally often in its past solutions, so naming either shortens the record of its past work by the same amount, and suppose each, added to the learner's vocabulary, lets it solve the same number of new problems directly. The learner then keeps working: it searches for solutions to a curriculum of harder problems, names recurring pieces of the solutions it finds, and searches again. Whether the first abstraction was in its vocabulary during that work can change which problems it reaches, which pieces it names next and therefore what it can do at the end. Two abstractions with the same record can leave the learner with different futures.

This paper studies that difference. We call the effect of having an abstraction during later discovery its discovery value, and an abstraction with large positive discovery value fertile, after the older use of fertility for theories that keep generating new problems and results (McMullin 1976; Kuhn 1977). The question is when learning an abstraction changes what a bounded learner discovers next, by how much, and whether the change can be predicted from what the learner knows when it adopts the abstraction. Every part of the general idea has been studied: abstractions that make later abstractions easier to find are the mechanism of library-learning program synthesizers (Ellis et al. 2021; Bowers et al. 2023), and a computational model with human experiments shows that the order in which concepts are acquired changes which later concepts people reach (Zhao et al. 2024). What is missing is a way to separate the effect of an abstraction on later discovery from

its effect as a tool, under equal computational budgets, and an account of what that separation implies for choosing abstractions.

We make four contributions. First, we define the discovery value of an abstraction through an early-, late- and no-access design. The abstraction is available at the final test in both compared conditions, so the contrast isolates its effect on what was learned. Second, we prove that discovery value is not determined by an abstraction's compression of past solutions, its direct usefulness or the immediate effect of adding it to the learner's search. Third, we prove that the ordering of two abstractions by discovery value can reverse when verifying candidate programs becomes more expensive, so that no score computed without reference to the search budget can order abstractions correctly at every cost. Fourth, we measure discovery value for 2,400 candidate abstractions in 300 randomly generated worlds in each of two domains, list-manipulating programs and navigation between rooms. In both, discovery value varies more across abstractions than direct value does, takes both signs, and is reliable across independent test problems; the compression of past solutions predicts it barely or not at all, direct value predicts it with the wrong sign, and the best predictor we tested, a costly look-ahead on the learner's own curriculum, recovers about a third of the attainable benefit in one domain and little in the other. Rankings of abstractions by discovery value change with verification cost far more than measurement noise allows.

The setting is program search with a growing library. The mechanism resembles what accounts of imagination describe when a mind constructs possibilities that change its own later construction of possibilities, but nothing here concerns imagery or experience, and the results are about the learners and worlds simulated.

2. Prior work

2.1 Macro-operators and the utility problem

Planning research learned early that stored solution fragments can speed later search. STRIPS generalized successful plans into macro-operators that could be reused as single steps (Fikes et al. 1972). Korf showed that macro-operators learned for permutation puzzles such as the Eight Puzzle and Rubik's Cube allow solutions to be assembled without search, and treated macro-operator learning as a general weak method (Korf 1985). Iba selected macro-operators from solution traces with heuristics and filtered them by their usefulness (Iba 1989), Soar's chunking summarized the results of subgoal processing into new rules (Laird et al. 1986), and Macro-FF extracted, ranked and filtered macro-operators from experience in a domain to speed later planning (Botea et al. 2005). In reinforcement learning, options extend primitive actions with temporally extended ones (Sutton et al. 1999). Whether options help depends on the problem: Jong, Hester and Stone found that standard ways of adding temporal abstraction help only in limited contexts and traced their benefit to the exploration they induce (Jong et al. 2008), and Solway and colleagues gave a Bayesian account of which hierarchies of subtasks are optimal for a set of tasks and found that people tend to discover such hierarchies (Solway et al. 2014).

The same literature identified the cost side. Minton quantified the utility problem: learned control knowledge slows a problem solver when the cost of matching and using it exceeds what it saves (Minton 1990). Tambe, Newell and Rosenbloom traced slowdowns in Soar to expensive chunks (Tambe et al. 1990), and Markovitch and Scott treated the decision of which acquired knowledge to keep as a family of selection mechanisms (Markovitch and Scott 1993). All of this concerns an abstraction's direct effect on the search that uses it, which in the terms below is its direct value together with its branching cost.

2.2 Library learning and self-improving search

Algorithmic probability weights programs by their length (Solomonoff 1964), and Levin's universal search allocates search time to programs according to that weight (Levin 1973). The optimal ordered problem solver applies this to a sequence of problems and reuses code from earlier solutions when doing so pays (Schmidhuber 2004); PowerPlay searches for new problems together with modifications of the solver that solve them without losing earlier skills (Schmidhuber 2013). Baxter's model of inductive bias learning shows how a learner embedded in an environment of related tasks can learn a hypothesis space that serves new tasks from the same environment (Baxter 2000). Dyna-style agents improve a policy with experience simulated from a learned model (Sutton 1990); there too internally generated search improves behaviour, with a fixed vocabulary of actions.

Library learning in program synthesis makes the loop concrete. The exploration-compression algorithm alternated between searching for programs and compressing the solutions into reusable components (Dechter et al. 2013). DreamCoder alternates between solving tasks, refactoring solutions into a growing library of abstractions and training a neural search policy, and its libraries deepen as later abstractions are built from earlier ones (Ellis et al. 2021, 2023). Stitch synthesizes abstractions by corpus-guided top-down search and evaluates library quality by compressivity (Bowers et al. 2023); babble finds abstractions modulo an equational theory (Cao et al. 2023). Cropper showed that forgetting learned programs can reduce the hypothesis space and the sample complexity of an inductive logic programming learner (Cropper 2020), a direct instance of the branching cost. Berlot-Attwell and colleagues re-examined three in-context library-learning systems built on large language models and found that their gains often vanished or reversed when baselines received the same computational budget, and found no evidence that one system reused its learned lemmas (Berlot-Attwell et al. 2026).

Recent work on self-improving agents separates the skills an agent uses from the procedure that improves them. MetaSkill-Evolve evolves both on two timescales (Wang et al. 2026). Hindsight meta-experience distillation re-executes an incumbent and a revised improvement procedure from the same restored discovery state, so that the effect of a revision on the subsequent search can be observed under a shared condition (Feng et al. 2026). That comparison and the early- and late-access contrast below share a logic: hold the starting state fixed and compare the discovery processes two choices induce.

2.3 Abstraction learning in people

Which primitives a learner starts with changes what it learns: Piantadosi, Tenenbaum and Goodman showed that different primitive sets in a compositional language of thought predict different learning curves for rule-based concepts (Piantadosi et al. 2016), and program-induction models learn new concepts by reusing parts of old ones (Lake et al. 2015). Zhao, Lucas and Bramley modelled conceptual bootstrapping with a repertoire that caches and reuses elements of earlier insights; the model predicts different learned concepts when the same evidence arrives in different orders, and four experiments with 570 participants showed the predicted curriculum-order and garden-pathing effects (Zhao et al. 2024). Hernandez Cano and colleagues argue that existing library-learning algorithms compress past tasks retrospectively, and report that people building geometric patterns choose abstractions in a way consistent with compressing future tasks under a non-stationary task-generating process (Hernandez Cano et al. 2026). Resource-rational analysis supplies the general frame in which such choices trade benefits against computational costs (Lieder and Griffiths 2020).

2.4 Novelty, creativity and usable information

Boden distinguished combinational, exploratory and transformational creativity (Boden 1998). Wiggins formalized creative systems as search in a conceptual space governed by rules for traversal and evaluation (Wiggins 2006), Ritchie examined whether transforming a conceptual space can be made precise (Ritchie 2006), and Espírito Santo, Wiggins and Cardoso use formal learning theory to argue that novelty is in general neither necessary nor sufficient for transformational creativity (Espírito Santo et al. 2024). Schmidhuber's theory of creativity rewards an agent for improvements in its own ability to predict or compress (Schmidhuber 2010). Tria and colleagues modelled correlated novelties, in which each novelty opens the space for further ones (Tria et al. 2014). Xu and colleagues defined usable information relative to a computationally bounded observer; unlike Shannon information, it can be created by computation (Xu et al. 2020). Our setting has the same structure: an abstraction defined from existing primitives adds nothing to what can be expressed, but changes what a bounded search can reach.

2.5 Fertility as a criterion for theories

Philosophers of science have long counted fruitfulness among the virtues of theories. Carnap required that an explication be fruitful, useful for formulating many universal statements (Carnap 1950); Kuhn listed fruitfulness with accuracy, consistency, scope and simplicity as shared criteria of theory choice (Kuhn 1977); Lakatos judged research programmes by whether they keep predicting novel facts (Lakatos 1970); and McMullin examined fertility, a theory's capacity to be developed fruitfully, and argued that it makes the theory as developed over time the proper unit of appraisal (McMullin 1976). These accounts concern scientific theories and human judgment. The discovery value defined here is a measurable counterpart for abstractions in a bounded learner, and the results below bear on one of their themes: whether fertility can be assessed when an idea is adopted or only after its consequences have been worked out.

3. Bounded search over a growing library

3.1 Expressibility and accessibility

Let X be a finite set of states and $\Sigma = \{p_1, \dots, p_b\}$ a set of primitive transformations $p_i : X \to X$. A library $L \supseteq \Sigma$ is an ordered set of tokens, each defined by an expansion $\delta(t) \in \Sigma^+$, a nonempty sequence of primitives; a primitive is its own expansion, and $|t|_\Sigma$ is the length of the expansion. A program $\pi = t_1 \cdots t_m$ is a nonempty sequence of tokens, applied from left to right, and its behaviour is the composed map $f_\pi = t_m \circ \cdots \circ t_1$. The expressible set of L is $H(L) = \{f_\pi : \pi \in L^+\}$, and the description length $\ell_L(h)$ of a behaviour $h \in H(L)$ is the length of the shortest program with behaviour h. A task specifies which behaviours are acceptable: in the first domain of Section 5 exactly one behaviour on all of X, in the second any behaviour that takes a given start state to a given goal.

The search procedure enumerates programs in length-lexicographic order: all programs of length 1, then all of length 2, and so on, ordered within a length by the token order. Each candidate costs

$$\kappa(t) = c_v + c_x \, |t|_\Sigma,$$

where t is its last token: c_v is the cost of checking a candidate against the task and $c_x|t|_\Sigma$ the cost of executing the last token, with the result of the prefix cached. A task is solved within budget B if the first program meeting its specification is reached at cumulative cost at most B. The first program found is a shortest one over the current library. We write $C_L(h)$ for the cumulative cost at which behaviour h is first produced and $A_B(L) = \{h : C_L(h) \le B\}$ for the accessible set. With $n = |L|$ tokens and $c_x = 0$, the number of programs of length at most d is

$$N_n(d) = \sum_{j=1}^{d} n^j.$$

Proposition 1 (expressibility and accessibility). Let a be a token with $\delta(a) \in \Sigma^+$, added last in the token order. (i) $H(L \cup \{a\}) = H(L)$. (ii) If $c_x = 0$, a behaviour with description length ℓ over n tokens is first produced at a cost between $c_v(N_n(\ell - 1) + 1)$ and $c_v N_n(\ell)$. (iii) If no shortest program for h over $L \cup \{a\}$ contains a and $\ell_L(h) \ge 2$, then $C_{L\cup\{a\}}(h) > C_L(h)$, so h leaves the accessible set for every budget in $[C_L(h), C_{L\cup\{a\}}(h))$. If instead adding a shortens the description of h from ℓ to $\ell' < \ell$ and $c_x = 0$, then h enters the accessible set for every budget in $[c_v N_{n+1}(\ell'), c_v(N_n(\ell - 1) + 1))$ when that interval is nonempty.

Proof. (i) Replacing each occurrence of a by its expansion turns a program over $L \cup \{a\}$ into a program over L with the same behaviour, and $L \subseteq L \cup \{a\}$. (ii) All programs shorter than ℓ come first, at unit cost c_v each. (iii) Let π be the first program for h over L, of length $m = \ell_L(h) \ge 2$. A shortest program for h over $L \cup \{a\}$ does not contain a, so it is a program over L and has length at least m; hence $\ell_{L\cup\{a\}}(h) = m$, and every program of length m for h over $L \cup \{a\}$ is a program over L. The lexicographic order of sequences over the old tokens is unchanged, so π is still the first program for h, and every candidate examined before π over L is still examined before it. The

one-token program a is also examined before π, at positive cost. So h is first produced by π at a strictly greater cost. The second statement follows from (ii). □

These facts are elementary, and the first is the reason Section 4 can treat discovery value as a matter of access: an abstraction defined from primitives never changes what can be expressed. The second part of (iii) is the toy calculation behind macro-operators, under which an abstraction helps when $\ell' \log(n+1) < \ell \log n$ roughly. The first part is its cost: every behaviour that needs at least two tokens and has no shortest program using the abstraction is reached later. Figure 1 shows both effects in the list domain of Section 5. Adding one four-primitive abstraction, the most compressive pattern in one world's history, barely changes the number of distinct behaviours a budget reaches, but at a budget of 13,082 units it makes 1,356 behaviours newly reachable and 1,295 unreachable, out of 3,011 reachable without it.

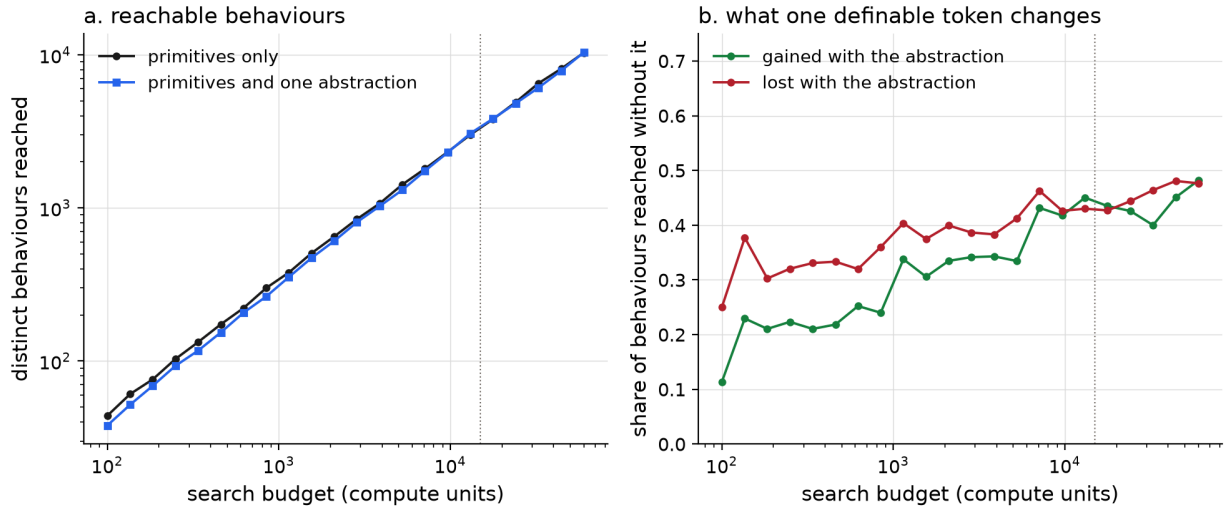


Figure 1: Behaviours reachable by length-lexicographic search in the list domain, with the six primitives alone and with one added abstraction (rev cpy rot swp), as a function of the budget, with verification cost 1 and execution cost 1 per primitive. (a) Number of distinct behaviours reached. (b) Behaviours gained and lost when the abstraction is added, as shares of the number reached without it. The dotted line marks the budget used in the experiments.

3.2 Discovery and the three values of an abstraction

A discovery process takes a library L and a curriculum of discovery tasks. In each of three rounds it searches for every task with budget B, compresses the solutions found and adds the resulting abstractions to the library. Compression is greedy description-length reduction in the manner of Stitch (Bowers et al. 2023) and of minimum description length more generally (Rissanen 1978): a contiguous pattern of k tokens occurring c times without overlap in the solutions saves

$$g = c\,(k-1) - k$$

tokens, the shortened occurrences minus the definition; the compressor adds the pattern with the largest saving among patterns of two or three tokens that occur at least twice, rewrites the solutions, and repeats once, skipping patterns whose behaviour is the identity or duplicates a token. New abstractions may contain earlier ones, so libraries deepen.

For a candidate abstraction a, let K_1 be the abstractions discovered from $L \cup \{a\}$ and K_0 those discovered from L, and let $U(M)$ be the share of withheld test tasks that a frozen search with library M solves within an evaluation budget. Learned tokens are placed in a fixed order at evaluation, by primitive length and then expansion, so that identical learned sets give identical libraries. Table 1 sets out the three conditions.

Table 1: Table 1. The early-, late- and no-access design. Each discovery run gives every task the same budget, so the compared conditions use the same search allowance.

condition	a during discovery	a at evaluation	library evaluated
no access	no	no	$L \cup K_0$
late access	no	yes	$L \cup \{a\} \cup K_0$
early access	yes	yes	$L \cup \{a\} \cup K_1$

The direct value, discovery value and total value of a are

$$V_{\text{direct}}(a) = U(L \cup \{a\} \cup K_0) - U(L \cup K_0), \qquad V_{\text{disc}}(a) = U(L \cup \{a\} \cup K_1) - U(L \cup \{a\} \cup K_0),$$

and $V_{\text{total}}(a) = V_{\text{direct}}(a) + V_{\text{disc}}(a)$. Both terms of V_{disc} contain a, so a difference cannot come from one evaluator holding the original tool and the other not. Both conditions also pay the same cost to acquire a, so the contrast compares timing; a claim that adopting a is worth its acquisition cost requires a further comparison with no access. Discovery value is relative to the learner, the discovery and evaluation budgets, the curriculum and the distribution of later tasks; it is not a property of the abstraction alone.

Because evaluation depends only on the learned set, $V_{\text{disc}}(a) \neq 0$ requires $K_1 \neq K_0$. Early access can change the learned set in two ways: the learner can build descendants, abstractions that contain a, or it can learn different abstractions that do not contain a because a changed which tasks it solved and how it wrote their solutions.

4. What compression and direct use cannot determine

The two constructions in this section use a free domain over $\Sigma = \{0, 1, 2, 3\}$ in which a program's output is the concatenation of its tokens' expansions, so two programs are equivalent exactly when their expansions are equal; this is a transformation domain in which each token appends its expansion to a string. Candidates cost one unit each ($c_v = 1$, $c_x = 0$), budgets count candidates, and the

discovery process is the one of Section 3.2. Both constructions are checked by exact enumeration in the accompanying code.

4.1 A separation

Proposition 2. There are a history, a curriculum of two task families, a set of test tasks, a budget and two abstractions a and a' of the same length such that a and a' have the same compression saving and frequency in the history, the same direct value, the same number of discovery tasks newly solved in the first round and the same frontier compression (defined in Section 4.3), while $V_{\text{disc}}(a) = 1/3$ and $V_{\text{disc}}(a') = 0$.

Construction and proof. Let $\sigma(x) = 3 - x$, applied symbolwise, and let $a = 012$ and $a' = \sigma(a) = 321$. The history is $\{0123, 3210, 012321, 321012\}$, which σ maps onto itself; each of a and a' occurs three times without overlap, so each has frequency 3 and saving $3 \cdot 2 - 3 = 3$. The curriculum has the family $F = \{aay : y \in \Sigma\}$ and its image $\sigma(F)$. The test tasks are $G = \{yaa : y \in \Sigma\}$, its image $\sigma(G)$ and $S = \{aaaay : y \in \Sigma\}$, twelve in all, and $B = 500$ for discovery and evaluation. With the four primitives alone, every discovery and test task has description length at least 7, and $N_4(6) = 5{,}460 > 500$, so nothing is solved: $K_0 = \emptyset$ and the no-access score is 0.

With a added as the fifth token, the task aay is the program (a, a, y), reached at cost $N_5(2) + 25 \cdot 4 + 5 \cdot 4 + y + 1 = 151 + y$, with the symbols read as token indices. The four solutions contain the pattern (a, a) four times, with saving $4 \cdot 1 - 2 = 2$, and no other pattern twice, so $K_1 = \{aa\}$; the next round finds nothing to compress. At evaluation with late access, yaa costs $N_5(2) + 25y + 24 + 1 = 55 + 25y \le 130$, while $aaaay$ needs five tokens and is first reachable after $N_5(4) = 780$ candidates, so $V_{\text{direct}}(a) = 4/12$. With early access the library has six tokens; $aaaay$ is the program (aa, aa, y) at cost $N_6(2) + 5 \cdot 36 + 5 \cdot 6 + y + 1 = 253 + y \le 256$, so the four tasks of S are added and $V_{\text{disc}}(a) = 4/12$. The map σ carries every statement about a to one about a' except the last: $\sigma(S)$ is not among the test tasks, so the descendant $a'a'$ only shortens the tasks of $\sigma(G)$, which late access already solves, and $V_{\text{disc}}(a') = 0$. In both cases the first round newly solves four tasks of a single family, so frontier compression, which counts only patterns recurring across families, is 0 for both. □

The difference between a and a' lies entirely in the later tasks: those tasks reuse two copies of a and never two copies of a'. Nothing measured on the history, on the abstraction's direct use or on the first round of discovery can register that difference, because the history, the direct uses and the first round are exchanged by σ. A predictor of discovery value has to bet on how later demands resemble the curriculum.

4.2 A reversal with verification cost

Proposition 3. There are a history, a curriculum, test tasks, an evaluation budget and two abstractions a and b such that $V_{\text{disc}}(a) > V_{\text{disc}}(b)$ when the discovery budget is 1,000 candidates and $V_{\text{disc}}(a) < V_{\text{disc}}(b)$ when it is 200.

Construction and proof. Let $a = 012$ and $b = 3210$, with history, curriculum and test tasks

$$\{a3b,\ b1a,\ ab\} = \{01233210,\ 32101012,\ 0123210\},$$

$$\{yaaz : (y, z) \in \{(0, 1), (1, 2), (2, 3), (3, 0)\}\} \cup \{bby : y \in \Sigma\},$$

$$\{aaaay : y \in \Sigma\} \cup \{bbbby : y \in \{0, 1\}\},$$

and an evaluation budget of 300. In the history a and b each occur three times without overlap, so their compression savings are $3 \cdot 2 - 3 = 3$ and $3 \cdot 3 - 4 = 5$. Without either abstraction every discovery task has at least eight symbols and nothing is solved. With a, the task $yaaz$ is a four-token program, reached after at most $N_5(4) = 780$ candidates; at a discovery budget of 1,000 all four are solved and $K_1 = \{aa\}$, as in Proposition 2. At a budget of 200 none is solved, since the first of them, $0aa1$, comes at $N_5(3) + 122 = 277$, and $K_1 = \emptyset$. With b, the task bby is a three-token program reached at $151+y \leq 154$, so at either budget $K_1 = \{bb\}$. At evaluation, late access solves no test task for either abstraction, since a five-token program needs more than 780 candidates; early access solves the four tasks $aaaay$ with aa and the two tasks $bbbby$ with bb, each as a three-token program at cost at most 256. So at a discovery budget of 1,000, $V_{\text{disc}}(a) = 4/6 > V_{\text{disc}}(b) = 2/6$, and at 200, $V_{\text{disc}}(a) = 0 < 2/6$. □

A budget of 200 candidates in place of 1,000 is what a fivefold increase in the cost of checking a candidate does to a fixed amount of compute. The fertility of a depends on discovery programs of four tokens and that of b on programs of three, and the deeper one is lost first.

Corollary. A score assigned to each abstraction without reference to the discovery budget orders a and b the same way at both budgets, so it misorders them at one of the two. In this construction the compression savings are 3 for a and 5 for b, which places b first and is wrong at the higher budget.

Retrospective compression, reuse frequency and the length of an abstraction are all scores of this kind. The corollary does not say that such scores carry no information about discovery value; it says that any relation between them and discovery value must change with the budget, and Section 5.5 measures how much it changes.

4.3 Estimating discovery value from the learner's own data

Proposition 2 shows that discovery value depends on later demands that the learner has not seen. A learner can still estimate it if the families in its curriculum resemble later families, for example if both are drawn from the same process. Discovery value also acts only through the learned set, which changes only through the discovery tasks that a brings within reach or pushes out of reach and through the way their solutions are written. These two observations suggest a ladder of predictors, ordered by what they use and how much they cost:

- history compression, the saving g of a in the history from which candidates are drawn, and history frequency, its number of occurrences there;

- direct value, measured on half of the test families and compared with discovery value on the other half, so that no test task enters both; it uses information a learner would not have and serves as a reference;
- frontier yield, the change in the number of discovery tasks solved in the first round when a is added, and frontier gain, the number of discovery tasks solved in the first round with a and not without it;
- frontier compression, the saving that the learner's compressor finds in the first-round solutions with a, counting only patterns that recur in at least two task families, minus the same quantity without a;
- look-ahead, the design of Table 1 applied to the learner's own curriculum: discovery on half of the families with and without a, evaluation on the other half, averaged over both directions.

The frontier statistics cost one round of search with a. The look-ahead costs two discovery runs and four evaluations per candidate, which is comparable to measuring discovery value itself, but uses no test task.

5. Experiments in two domains

5.1 Domains and protocol

The first domain consists of transformations of lists of length 5 over $\{0, 1, 2\}$, so X has 243 states. The six primitives rotate the list left by one (rot), reverse it (rev), swap the first two elements (swp), add one modulo 3 to the first element (inc), copy the first element into the second position (cpy), and replace the first element by the sum of the first two modulo 3 (add). Each world draws five level-1 concepts, sequences of two to four primitives with no shorter primitive description and distinct behaviours, and four level-2 concepts, sequences of two or three units of which each is a level-1 concept with probability 0.75 and a primitive otherwise, with at least two concepts. A task family is a template of two or three slots, each a level-2 concept with probability 0.45, a level-1 concept with probability 0.25 or a free slot; every template has at least one fixed and one free slot, and its tasks fill the free slots with random primitives. A task asks for the exact behaviour on all 243 lists. The learner's history is 30 programs, each a sequence of two or three level-1 concepts, each followed by a random primitive with probability 0.4.

The second domain is navigation in a building of 4×4 rooms, each of 3×3 cells, joined by single-cell doors placed along a random spanning tree of the room grid plus each remaining adjacency with probability 0.35; a world has about 160 cells. The primitives move one cell north, east, south or west and leave the agent in place at a wall. A family is an ordered pair of rooms at most two rooms apart, and its tasks ask for a plan taking a start cell in the first room to a goal cell in the second, at least four moves away; only the start cell is checked, so many programs solve a task. The history is the shortest plan for each of 30 random trips of three to six moves. The two domains differ in their primitives, in whether a task fixes a whole behaviour or one outcome, and in where

their regularities come from: a generative grammar of concepts in one, the geometry of a building in the other.

In both domains a world has 24 discovery families and 60 disjoint test families of four tasks each, 96 discovery and 240 test tasks. The candidates are the eight patterns of two to four primitives in the history with the largest compression saving, keeping one per behaviour and excluding the identity and single primitives. The base library is the primitives. Every search has a budget of 15,000 units, with execution cost $c_x = 1$ per primitive; evaluation uses verification cost $c_v = 1$, and discovery is run at $c_v \in \{1, 2, 4, 8\}$. Discovery has three rounds of two abstractions each. We generated 300 worlds per domain, which gives 2,400 candidates per domain and cost. Agreement between a predictor and discovery value is the Spearman correlation across the eight candidates of a world, averaged over worlds, with 95% intervals from 4,000 bootstrap resamples of worlds. Reliability is the same correlation between discovery values measured on two halves of the test families. The design, parameters and predictor definitions were settled on pilot worlds drawn from other seeds; the curriculum was enlarged from 12 to 24 families during piloting because the smaller one gave noisier outcomes, and Section 5.4 reports the smaller one as a check. The reported run uses fresh worlds.

At verification cost 1, the base learner solves on average 29.3 of the 96 discovery tasks in the first round and 41.7 in its last round in the list domain, and 23.7 and 31.2 in navigation. Its final library solves 40.9% of the test tasks in the list domain and 27.7% in navigation.

5.2 Discovery value is large, two-signed and reliable

Figure 2 plots discovery value against direct value for every candidate at verification cost 1. In the list domain the standard deviation of discovery value across candidates is 0.096, against 0.042 for direct value: early access changes the share of test tasks solved by more than five percentage points for 47.8% of candidates, in either direction, with 46.2% of candidates positive and 41.8% negative. Discovery value exceeds direct value in magnitude for 65.6% of candidates, and the two have opposite signs for 55.8% of those for which both are nonzero. In navigation the corresponding standard deviations are 0.053 and 0.037; 32.0% of candidates change the test score by more than five points through discovery, and the signs are opposite for 58.3%.

These differences are not noise in the test set. The split-half reliability of discovery value across candidates within a world is 0.64 (95% interval 0.60 to 0.68) in the list domain and 0.47 (0.44 to 0.51) in navigation, or 0.78 and 0.64 for the full test set by the Spearman–Brown formula. The reliable differences also survive matching on the quantities a learner usually sees. Among 505 pairs of candidates in the same list world whose compression savings differ by at most 1 and whose direct values, measured on the first half of the test families, differ by at most one task, the difference in discovery value measured on one quarter of the test families correlates at 0.57 with the difference measured on another quarter, and 40.6% of these pairs differ by more than five points; in navigation, 912 such pairs give 0.30 and 38.4%. Proposition 2 constructs such a pair by hand; the random worlds contain many.

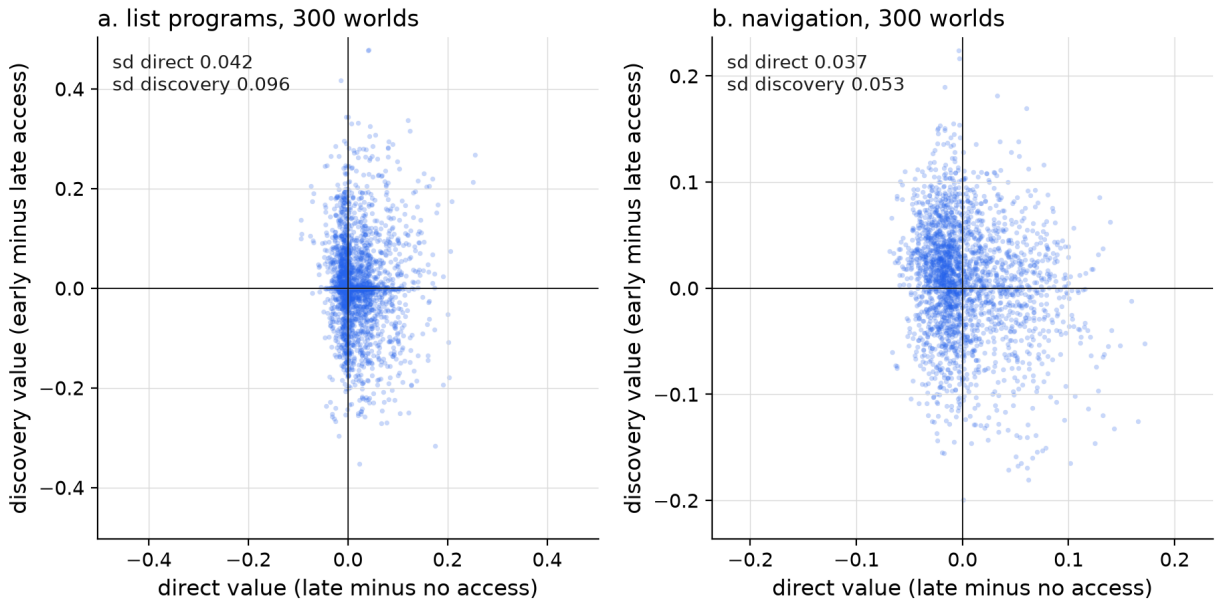


Figure 2: Direct value against discovery value for every candidate abstraction at verification cost 1, 2,400 candidates per domain from 300 worlds. Points are jittered slightly. (a) List programs. (b) Navigation.

Direct value and discovery value are negatively related. Measured on disjoint halves of the test families, their within-world correlation is -0.12 (95% interval -0.16 to -0.07) in the list domain and -0.23 (-0.26 to -0.20) in navigation. In individual worlds we inspected, one mechanism is visible. When an abstraction is enough by itself to bring a group of test tasks within reach, late access already collects that gain and early access has little to add, whereas a fragment of the same structure may solve nothing directly and still let the learner build the larger abstraction during discovery. The total effect of having the abstraction early is split between the two channels in proportions that vary from abstraction to abstraction.

5.3 How early access changes what is learned

Early access almost always changes the learned library: the learned set is identical with and without the candidate for 5.3% of candidates in the list domain and 1.4% in navigation, and in all those cases discovery value is exactly zero, as Section 3.2 requires. For 74.7% of list candidates and 78.5% of navigation candidates the learner builds at least one descendant, an abstraction containing the candidate. The remaining 21.3% and 20.2% change the learned set without any descendant: the candidate changes which discovery tasks are solved and how their solutions are written, and the compressor then selects different abstractions. These redirections carry 19.7% and 19.1% of the total absolute discovery value. In the list domain their mean effect is negative, -0.024, against 0.016 for candidates with descendants, and the number of descendants correlates with discovery value at 0.15 (0.10 to 0.21) within worlds; in navigation the correlation is indistinguishable from zero (-0.05 to 0.05). Figure 4b shows the two distributions.

Being one of the world's level-1 concepts helps a candidate only slightly. True concepts make up 47.3% of the candidates and have mean discovery value 0.011, against 0.003 for fragments and cross-boundary patterns, and mean direct value 0.028 against 0.012. A ground-truth score that credits a candidate for supplying part of a concept the base learner never learned, weighted by how many test families use that concept, does not predict discovery value: its within-world correlation is −0.03 (−0.08 to 0.03).

5.4 How well discovery value can be predicted

Figure 3 gives the agreement of each predictor with discovery value. In the list domain at verification cost 1, history compression correlates with discovery value at 0.11 (0.06 to 0.15) and history frequency at 0.01; direct value, measured on the other half of the test families, at −0.12; frontier yield at 0.17 (0.12 to 0.22), frontier gain at 0.13 and frontier compression at 0.06; and the look-ahead at 0.22 (0.17 to 0.26). A predictor that measured each candidate's discovery value without error would be expected to correlate at about 0.88 with the measured values, given their reliability. In navigation, history compression correlates at 0.08 (0.04 to 0.12), frontier yield at 0.07, frontier gain at 0.03, frontier compression at −0.04 and the look-ahead at 0.08 (0.03 to 0.12), against a ceiling of about 0.80; direct value correlates at −0.23.

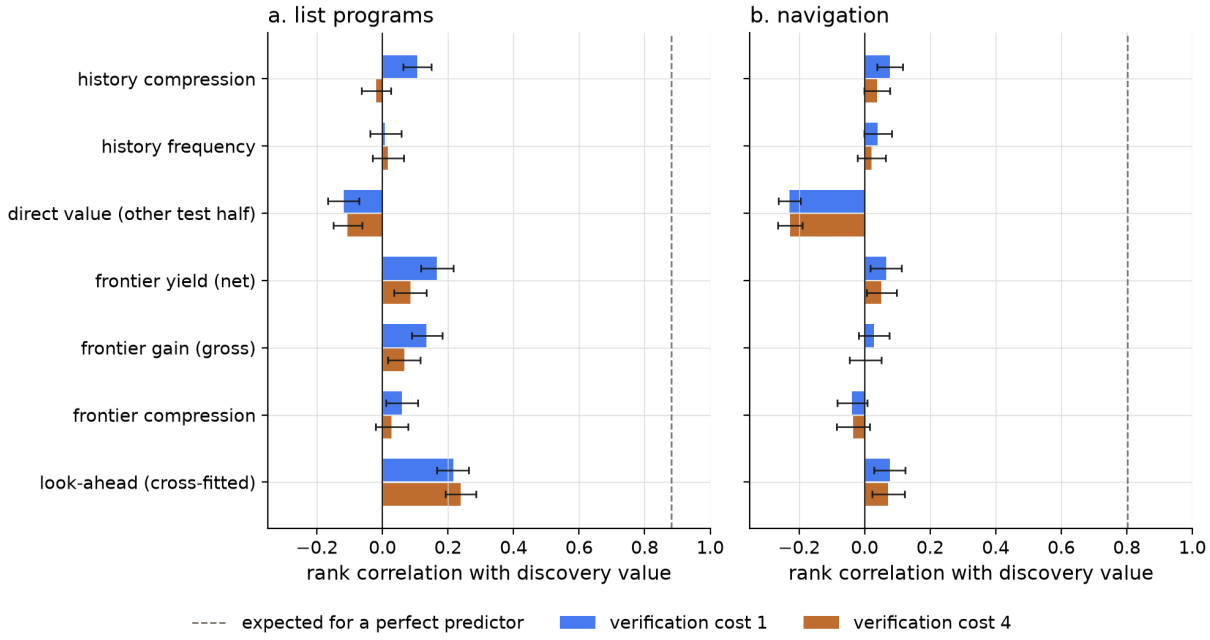


Figure 3: Within-world rank correlation between each predictor and discovery value, with 95% bootstrap intervals over 300 worlds, at verification costs 1 and 4. The dashed line is the correlation expected for a predictor that measured discovery value without error, given the reliability of the measured values at cost 1. Direct value is measured on half of the test families and compared with discovery value on the other half. (a) List programs. (b) Navigation.

In practical terms, choosing the candidate a predictor ranks first raises discovery value above the world's average candidate by 0.036 with the look-ahead, 0.022 with frontier yield and 0.013 with

history compression in the list domain, against 0.114 for choosing the best candidate in hindsight; the look-ahead thus captures about a third of the attainable gain. In navigation the look-ahead and history compression each gain 0.006 against an attainable 0.066, and choosing by direct value loses 0.016.

The ordering of predictors is stable when the curriculum is halved to 12 families in the list domain: history compression correlates at 0.10, frontier yield at 0.21, frontier gain at 0.19 and the look-ahead at 0.21, and direct value at -0.13.

5.5 Verification cost

Proposition 3 implies that rankings by discovery value can change with the cost of verification. Figure 4a measures how much. The agreement between rankings at verification cost 1 and at a higher cost, measured on disjoint halves of the test families and divided by the geometric mean of the two within-cost reliabilities, falls from 0.56 at cost 2 to 0.36 at cost 4 and 0.16 at cost 8 in the list domain, and from 0.58 to 0.32 and 0.19 in navigation. Rankings of the same candidates in the same worlds are thus close to unrelated when checking a candidate costs eight times as much, although the within-cost reliability stays between 0.57 and 0.64 in the list domain and between 0.46 and 0.48 in navigation.

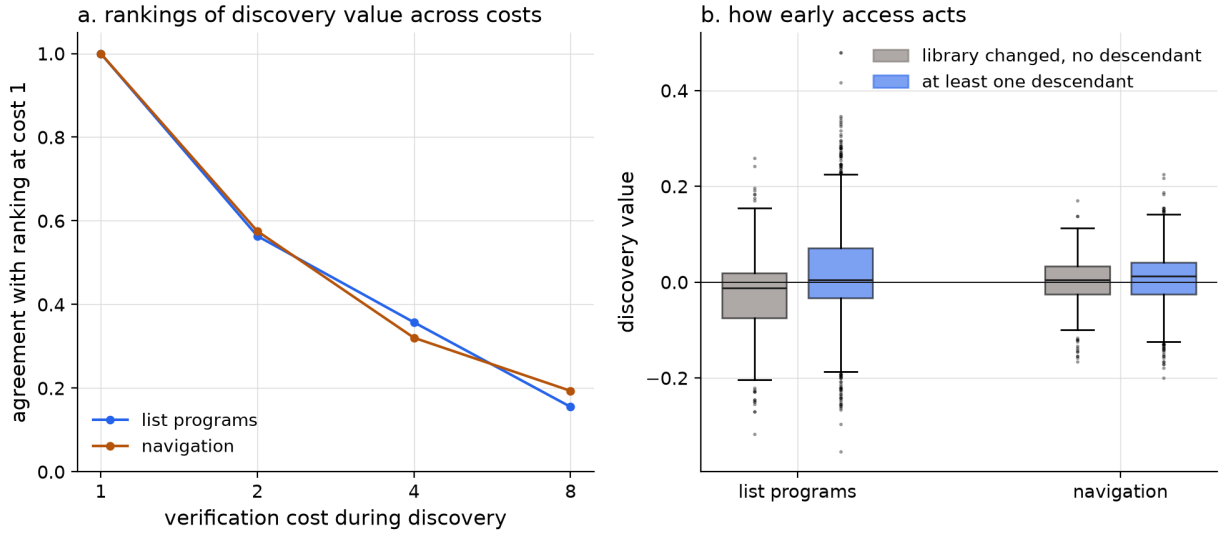


Figure 4: (a) Agreement between the ranking of candidates by discovery value at verification cost 1 and at higher costs during discovery, corrected for the reliability of each ranking. (b) Discovery value at verification cost 1 for candidates that changed the learned library without producing a descendant and for candidates with at least one descendant.

History compression does not depend on the verification cost, so its agreement with discovery value must move as the rankings move: in the list domain it is 0.11 at cost 1 and -0.02 (-0.06 to 0.03) at cost 4. A predictor that depends on the budget can follow the change only if it is computed at the right cost. The look-ahead does so in the list domain: computed at cost 1 and cost 4 and compared with discovery value at the same cost, it correlates at 0.23; compared with discovery

value at the other cost, at 0.15, a difference of 0.08 (0.05 to 0.11). The frontier statistics show no such advantage in the list domain, and in navigation only frontier yield shows a small one, 0.02 (0.00 to 0.04).

6. Limits and interpretation

The learner is deliberately simple. Its search enumerates programs without the observational-equivalence pruning used by practical enumerative synthesizers (Udupa et al. 2013; Albarghouthi et al. 2013; Gulwani et al. 2017), without a learned search policy of the kind DreamCoder trains (Ellis et al. 2021), and with a greedy compressor that adds two abstractions per round. Each of these choices affects which abstractions are learned and therefore discovery value. The worlds were generated by us, with regularities of two particular kinds, and the candidates were the most compressive patterns of a short history; a learner that proposes candidates differently faces a different set of comparisons. Nothing here measures an effect in people or in deployed systems.

Within those limits, the results separate two claims that are easy to run together. The first, that learned abstractions make later discovery easier, holds only weakly on average: mean discovery value is 0.007 in both domains. The second, that some abstractions are more fertile than others, is strongly supported: the differences between candidates are large, reliable across test families and present among candidates matched on compression and direct use. What the results do not support is that fertility can be read from the record an abstraction has when it is adopted. Compression of past solutions, the quantity library learners commonly optimize (Bowers et al. 2023; Cao et al. 2023), carries little information about discovery value in these worlds and none detectable in the list domain at verification cost 4, and direct usefulness points the wrong way. Prospective selection, which targets future tasks (Hernandez Cano et al. 2026), needs a model of those tasks, and Proposition 2 shows why: discovery value depends on demands beyond the learner's experience.

Several results would change this picture. A cheap predictor that tracked discovery value closely in either domain, a learner with equivalence pruning or a learned search policy for which compression did predict it, or worlds in which discovery value was small next to direct value would each narrow the claim. The look-ahead's partial success in the list domain and its failure in navigation also leave open what makes fertility estimable: in the list domain test families share concepts with the curriculum, while in navigation each pair of rooms poses its own route, and estimation from the curriculum depends on such sharing.

The philosophical accounts of fertility in Section 2.5 treat it as something judged over time, as a theory or research programme is developed. In these learners discovery value is a property of a trajectory: of the abstraction together with the learner's budget, curriculum and later demands. Among the eight most compressive patterns of each history, the abstractions a compression-based learner would adopt first, early access lowered the later score for 41.8% in the list domain and 38.2% in navigation. A learner that cares about what it will discover next has to spend computation looking ahead at its own curriculum, and the value it estimates holds only for the budget at which it looked.

The simulation code and a record of the run that produced every number and figure are in the repository; `simulation/run_all.py` reproduces them and runs 13 checks, including exact enumeration of the two constructions.

References

Albarghouthi, Aws, Sumit Gulwani, and Zachary Kincaid. 2013. "Recursive Program Synthesis." In *Computer Aided Verification: 25th International Conference, CAV 2013*, edited by Natasha Sharygina and Helmut Veith, vol. 8044. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-642-39799-8_67.

Baxter, Jonathan. 2000. "A Model of Inductive Bias Learning." *Journal of Artificial Intelligence Research* 12: 149–98. https://doi.org/10.1613/jair.731.

Berlot-Attwell, Ian, Tobias Sesterhenn, Frank Rudzicz, and Xujie Si. 2026. "Is This LLM Library Learning? Evaluation Must Account For Compute and Behaviour." *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3534–68. https://doi.org/10.18653/v1/2026.eacl-long.163.

Boden, Margaret A. 1998. "Creativity and Artificial Intelligence." *Artificial Intelligence* 103 (1-2): 347–56. https://doi.org/10.1016/s0004-3702(98)00055-1.

Botea, Adi, Markus Enzenberger, Martin Müller, and Jonathan Schaeffer. 2005. "Macro-FF: Improving AI Planning with Automatically Learned Macro-Operators." *Journal of Artificial Intelligence Research* 24: 581–621. https://doi.org/10.1613/jair.1696.

Bowers, Matthew, Theo X. Olausson, Lionel Wong, et al. 2023. "Top-Down Synthesis for Library Learning." *Proceedings of the ACM on Programming Languages* 7 (POPL): 1182–213. https://doi.org/10.1145/3571234.

Cao, David, Rose Kunkel, Chandrakana Nandi, Max Willsey, Zachary Tatlock, and Nadia Polikarpova. 2023. "babble: Learning Better Abstractions with E-Graphs and Anti-Unification." *Proceedings of the ACM on Programming Languages* 7 (POPL): 396–424. https://doi.org/10.1145/3571207.

Carnap, Rudolf. 1950. *Logical Foundations of Probability*. University of Chicago Press.

Cropper, Andrew. 2020. "Forgetting to Learn Logic Programs." *Proceedings of the AAAI Conference on Artificial Intelligence* 34 (04): 3676–83. https://doi.org/10.1609/aaai.v34i04.5776.

Dechter, Eyal, Jon Malmaud, Ryan P. Adams, and Joshua B. Tenenbaum. 2013. "Bootstrap Learning via Modular Concept Discovery." *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI 2013)*, 1302–9.

Ellis, Kevin, Catherine Wong, Maxwell Nye, et al. 2021. "DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning." *Proceedings of the 42nd ACM SIGPLAN*

International Conference on Programming Language Design and Implementation (PLDI 2021), 835–50. https://doi.org/10.1145/3453483.3454080.

Ellis, Kevin, Lionel Wong, Maxwell Nye, et al. 2023. "DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake–Sleep Bayesian Program Learning." *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 381 (2251): 20220050. https://doi.org/10.1098/rsta.2022.0050.

Espírito Santo, Luís, Geraint Wiggins, and Amílcar Cardoso. 2024. "Towards a Formal Creativity Theory: Preliminary Results in Novelty and Transformativeness." In *arXiv*, 2405.02148. Preprint. https://arxiv.org/abs/2405.02148.

Feng, Qianhan, Zhongzhen Huang, Yakun Zhu, Xiaofan Zhang, and Qi Dou. 2026. "Learning from Revision Consequences: Hindsight Meta-Experience Distillation for Self-Improving Agents." In *arXiv*, 2610.07979. Preprint. https://arxiv.org/abs/2610.07979.

Fikes, Richard E., Peter E. Hart, and Nils J. Nilsson. 1972. "Learning and Executing Generalized Robot Plans." *Artificial Intelligence* 3: 251–88. https://doi.org/10.1016/0004-3702(72)90051-3.

Gulwani, Sumit, Oleksandr Polozov, and Rishabh Singh. 2017. "Program Synthesis." *Foundations and Trends in Programming Languages* 4 (1-2): 1–119. https://doi.org/10.1561/2500000010.

Hernandez Cano, Leonardo, Ivan Zareski, Luisa El Amouri, et al. 2026. "Prospective Compression in Human Abstraction Learning." In *arXiv*, 2605.09985. Preprint. https://arxiv.org/abs/2605.09985.

Iba, Glenn A. 1989. "A Heuristic Approach to the Discovery of Macro-Operators." *Machine Learning* 3 (4): 285–317. https://doi.org/10.1007/bf00116836.

Jong, Nicholas K., Todd Hester, and Peter Stone. 2008. "The Utility of Temporal Abstraction in Reinforcement Learning." *Proceedings of the 7th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2008)*, 299–306.

Korf, Richard E. 1985. "Macro-Operators: A Weak Method for Learning." *Artificial Intelligence* 26 (1): 35–77. https://doi.org/10.1016/0004-3702(85)90012-8.

Kuhn, Thomas S. 1977. "Objectivity, Value Judgment, and Theory Choice." In *The Essential Tension: Selected Studies in Scientific Tradition and Change*. University of Chicago Press.

Laird, John E., Paul S. Rosenbloom, and Allen Newell. 1986. "Chunking in Soar: The Anatomy of a General Learning Mechanism." *Machine Learning* 1 (1): 11–46. https://doi.org/10.1007/bf00116249.

Lakatos, Imre. 1970. "Falsification and the Methodology of Scientific Research Programmes." In *Criticism and the Growth of Knowledge*, edited by Imre Lakatos and Alan Musgrave. Cambridge University Press. https://doi.org/10.1017/cbo9781139171434.009.

Lake, Brenden M., Ruslan Salakhutdinov, and Joshua B. Tenenbaum. 2015. "Human-Level Concept Learning Through Probabilistic Program Induction." *Science* 350 (6266): 1332–38. https://doi.org/10.1126/science.aab3050.

Levin, Leonid A. 1973. "Universal Sequential Search Problems." *Problems of Information Transmission* 9 (3): 265–66.

Lieder, Falk, and Thomas L. Griffiths. 2020. "Resource-Rational Analysis: Understanding Human Cognition as the Optimal Use of Limited Computational Resources." *Behavioral and Brain Sciences* 43: e1. https://doi.org/10.1017/s0140525x1900061x.

Markovitch, Shaul, and Paul D. Scott. 1993. "Information Filtering: Selection Mechanisms in Learning Systems." *Machine Learning* 10 (2): 113–51. https://doi.org/10.1007/bf00993503.

McMullin, Ernan. 1976. "The Fertility of Theory and the Unit for Appraisal in Science." In *Essays in Memory of Imre Lakatos*, edited by Robert S. Cohen, Paul K. Feyerabend, and Marx W. Wartofsky, vol. 39. Boston Studies in the Philosophy of Science. D. Reidel. https://doi.org/10.1007/978-94-010-1451-9_25.

Minton, Steven. 1990. "Quantitative Results Concerning the Utility of Explanation-Based Learning." *Artificial Intelligence* 42 (2-3): 363–91. https://doi.org/10.1016/0004-3702(90)90059-9.

Piantadosi, Steven T., Joshua B. Tenenbaum, and Noah D. Goodman. 2016. "The Logical Primitives of Thought: Empirical Foundations for Compositional Cognitive Models." *Psychological Review* 123 (4): 392–424. https://doi.org/10.1037/a0039980.

Rissanen, Jorma. 1978. "Modeling by Shortest Data Description." *Automatica* 14 (5): 465–71. https://doi.org/10.1016/0005-1098(78)90005-5.

Ritchie, Graeme. 2006. "The Transformational Creativity Hypothesis." *New Generation Computing* 24 (3): 241–66. https://doi.org/10.1007/bf03037334.

Schmidhuber, Jürgen. 2004. "Optimal Ordered Problem Solver." *Machine Learning* 54 (3): 211–54. https://doi.org/10.1023/b:mach.0000015880.99707.b2.

Schmidhuber, Jürgen. 2010. "Formal Theory of Creativity, Fun, and Intrinsic Motivation (1990–2010)." *IEEE Transactions on Autonomous Mental Development* 2 (3): 230–47. https://doi.org/10.1109/tamd.2010.2056368.

Schmidhuber, Jürgen. 2013. "PowerPlay: Training an Increasingly General Problem Solver by Continually Searching for the Simplest Still Unsolvable Problem." *Frontiers in Psychology* 4: 313. https://doi.org/10.3389/fpsyg.2013.00313.

Solomonoff, Ray J. 1964. "A Formal Theory of Inductive Inference. Part I." *Information and Control* 7 (1): 1–22. https://doi.org/10.1016/s0019-9958(64)90223-2.

Solway, Alec, Carlos Diuk, Natalia Córdova, et al. 2014. "Optimal Behavioral Hierarchy." *PLoS Computational Biology* 10 (8): e1003779. https://doi.org/10.1371/journal.pcbi.1003779.

Sutton, Richard S. 1990. "Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming." In *Machine Learning: Proceedings of the Seventh International Conference*, edited by Bruce W. Porter and Raymond J. Mooney. Morgan Kaufmann. https://doi.org/10.1016/b978-1-55860-141-3.50030-4.

Sutton, Richard S., Doina Precup, and Satinder Singh. 1999. "Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning." *Artificial Intelligence* 112 (1-2): 181–211. https://doi.org/10.1016/s0004-3702(99)00052-1.

Tambe, Milind, Allen Newell, and Paul S. Rosenbloom. 1990. "The Problem of Expensive Chunks and Its Solution by Restricting Expressiveness." *Machine Learning* 5 (3): 299–348. https://doi.org/10.1007/bf00117107.

Tria, Francesca, Vittorio Loreto, Vito D. P. Servedio, and Steven H. Strogatz. 2014. "The Dynamics of Correlated Novelties." *Scientific Reports* 4 (1): 5890. https://doi.org/10.1038/srep05890.

Udupa, Abhishek, Arun Raghavan, Jyotirmoy V. Deshmukh, Sela Mador-Haim, Milo M.K. Martin, and Rajeev Alur. 2013. "TRANSIT: Specifying Protocols with Concolic Snippets." *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 287–96. https://doi.org/10.1145/2491956.2462174.

Wang, Zefeng, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. 2026. "MetaSkill-Evolve: Recursive Self-Improvement of LLM Agents via Two-Timescale Meta-Skill Evolution." In *arXiv*, 2607.05297. Preprint. https://arxiv.org/abs/2607.05297.

Wiggins, Geraint A. 2006. "A Preliminary Framework for Description, Analysis and Comparison of Creative Systems." *Knowledge-Based Systems* 19 (7): 449–58. https://doi.org/10.1016/j.knosys.2006.04.009.

Xu, Yilun, Shengjia Zhao, Jiaming Song, Russell Stewart, and Stefano Ermon. 2020. "A Theory of Usable Information Under Computational Constraints." *International Conference on Learning Representations (ICLR 2020)*. https://arxiv.org/abs/2002.10689.

Zhao, Bonan, Christopher G. Lucas, and Neil R. Bramley. 2024. "A Model of Conceptual Bootstrapping in Human Cognition." *Nature Human Behaviour* 8 (1): 125–36. https://doi.org/10.1038/s41562-023-01719-1.